# Sentinel — 12-Week Execution Roadmap

**Team:** Person A (Analysis Engine / AI-ML) · Person B (Platform / Infra) **Goal:** Production-grade, defensible portfolio project with continuous, reviewable collaboration from both people every week, and genuine cross-training so each of you can speak to the *whole* system in an interview — not just your half.

---

## 0. How Cross-Training Works in This Plan

You explicitly don't want a hard wall between "A's stuff" and "B's stuff" — but the project's architecture genuinely does split cleanly along the AI/analysis vs. platform/infra line, and that split is *valuable* (it gives each of you a coherent "I built this" narrative). So instead of scrambling ownership, we layer cross-training on top of a stable primary-ownership structure, via four mechanisms used every week:

1. **Mandatory cross-review.** Every PR is reviewed by the *other* person before merge. The reviewer must leave at least one substantive comment (a question, a suggested alternative, a "why did you do X instead of Y") — not just "LGTM." This is the main vehicle for passive learning of the other half.
2. **Weekly Cross-Train Ticket (1–3 hrs each).** A small, self-contained task inside the *other person's* codebase — e.g., A adds a small read-only endpoint to B's gateway; B writes a Cypher query against A's graph and wires it into a script. Scoped small enough to never block the main work, but real enough to force hands-on contact.
3. **Swap Sprints (3 total, flagged below).** Half-day pairing sessions at the end of Phase 2, mid-Phase 3, and start of Phase 4, where each of you walks the other through extending your module live — literally driving the keyboard in the other's domain with the owner narrating.
4. **Architecture Decision Records (ADRs).** Every non-trivial design choice gets a short ADR (template in Section on repo structure) written by the owner, read and discussed by both before being marked "accepted." This builds shared mental models even where one person didn't write the code — and doubles as interview prep material later.

Each week below lists primary deliverables (your "I built this" core) *and* one cross-train ticket per person.

---

## Phase 1 — Foundations (Weeks 1–2)

**Week 1**

**Person A — Deliverables**

- Set up `services/analysis-engine` (Python, `uv` or Poetry, ruff/black, pytest, mypy in CI).
- Implement tree-sitter-based parser for **Python only**: walk a directory, parse each `.py` file, extract module-level imports, function defs (with line ranges), class defs (with methods).
- CLI entrypoint: `python -m analysis_engine.parse <repo_path>` → prints a JSON summary `{file, imports[], functions[], classes[]}`.
- Unit tests (5–8 fixture files) covering: relative imports, multi-line/aliased imports, nested classes, decorated functions, `__init__.py` edge cases.

**Person B — Deliverables**

- Monorepo skeleton + root `docker-compose.yml` bringing up: Postgres, Neo4j (with APOC + GDS plugins enabled), Redis, RabbitMQ.
- `services/gateway` (Fastify + TS): health-check endpoint, basic request logging middleware, Zod or Fastify-schema request validation scaffolding.
- `services/auth` (Fastify + TS + Prisma): `users` table migration, `/auth/register` and `/auth/login` returning a JWT (access token only for now).
- GitHub Actions CI: separate jobs for `analysis-engine` (lint+test+typecheck) and `gateway`/`auth` (lint+test+typecheck), triggered on every PR.

**Integration Point** Nothing flows end-to-end yet — but agree and write down (as ADR-0001) the **shared "RepoAnalysisJob" contract**: the fields a future job will need (repo URL, commit SHA, language list, output schema shape for parsed files). Both of you sign off on this even though it's unused — it constrains week 2+ design and prevents drift.

**Git Workflow**

- Branches: `feat/a-tree-sitter-python-parser`, `feat/b-monorepo-skeleton`, `feat/b-gateway-health`, `feat/b-auth-jwt`.
- Expect larger-than-usual PRs this week (scaffolding) — 1–2 PRs each, up to ~600–800 lines is fine since most is config/boilerplate.
- Cross-review: B reviews A's parser PR focusing on the **output JSON shape** (does it match ADR-0001?). A reviews B's `docker-compose.yml` and Prisma schema for the `users` table.

**Cross-Train Tickets**

- A: write the `Dockerfile` for `analysis-engine` (multi-stage, slim Python base) — reviewed by B.
- B: write one of A's parser unit tests from a spec A provides (e.g., "parser must handle `from . import x as y`") — reviewed by A.

**Learning Checklist**

- A: tree-sitter Python bindings + query syntax (3–4 hrs); Python packaging with `uv`/Poetry (1–2 hrs).
- B: Docker Compose multi-service networking (2–3 hrs); Fastify request lifecycle + schema validation (2–3 hrs); Prisma migrations basics (1–2 hrs).

**Demo-able Milestone** `docker-compose up` brings up all 5 infra containers cleanly. A CLI command parses a real small Python repo and prints a structured JSON tree of its files/functions/classes. `curl` against `/auth/register` and `/auth/login` returns a valid JWT. CI is green on both halves.

**Depth Check (AI-shortcut risk):** A generic AI-generated "docker-compose + auth skeleton" is genuinely commodity. The depth this week comes from **ADR-0001** (the contract design) and the **parser edge-case tests** — make sure those two artifacts exist and are non-trivial, not just the boilerplate.

---

**Week 2**

**Person A — Deliverables**

- Build a `ParsedRepo → GraphDocument` transformer: nodes for `File`, `Function`, `Class`; relationships `CONTAINS`, `IMPORTS` (file→file, resolved where possible).
- Neo4j ingestion script (`neo4j` Python driver, batched `UNWIND` writes — not row-by-row).
- Run end-to-end on one real small open-source Python repo (~50–100 files): parse → transform → load into Neo4j.
- Write 3 Cypher queries answering real questions: "files with highest fan-out", "files imported by ⩾5 others", "files with no imports/no importers (orphans)".
- ADR-0002: graph schema (node labels, relationship types, property naming conventions).

**Person B — Deliverables**

- Extend `auth` service: bcrypt password hashing, refresh-token issuance + rotation, refresh tokens table in Postgres.
- Postgres schema v1 (Prisma migration): `users`, `repos` (id, github_url, owner_id, default_branch, last_analyzed_at), `analysis_jobs` (id, repo_id, status enum, commit_sha, created_at, completed_at).
- Gateway: `/repos` endpoints — `POST /repos` (register a repo URL), `GET /repos/:id` — backed by `auth` for JWT verification.
- Register a GitHub OAuth App (manual step) + stub `/auth/github/callback` (doesn't need to do anything real yet — just receive the code and log it).

**Integration Point** The `repos` table schema (B) must match the "repo identity" fields A's ingestion script expects per ADR-0001 (repo URL, commit SHA). Concretely: B shares the Prisma schema for `repos`; A's ingestion script takes those same fields as CLI args/config — so when wired together in Week 5, no schema renegotiation is needed.

**Git Workflow**

- Branches: `feat/a-graph-transformer`, `feat/a-neo4j-ingestion`, `feat/b-auth-refresh-tokens`, `feat/b-postgres-schema-v1`, `feat/b-repos-endpoints`.
- ~2–3 PRs each, target <400 lines per PR from here on (scaffolding exception is over).
- Cross-review: A reviews the `repos`/`analysis_jobs` Prisma schema (A is the eventual consumer — sanity-check field names/types now). B reviews ADR-0002 graph schema doc — doesn't need deep Neo4j knowledge, just check naming consistency and whether it's extensible for multi-language later.

**Cross-Train Tickets**

- A: add a `GET /health/graph` endpoint to the gateway that runs a trivial Cypher count query and returns `{node_count, relationship_count}` — reviewed by B.
- B: open Neo4j Browser, run A's 3 Cypher queries against the loaded graph, and write a 1-paragraph summary of what each returns — reviewed by A.

**Learning Checklist**

- A: Neo4j data modeling + Cypher (4–5 hrs); batched writes with `UNWIND` (1 hr).
- B: bcrypt + refresh token rotation patterns (2 hrs); Prisma relations/enums (1–2 hrs); GitHub OAuth App registration flow (1 hr).

**Demo-able Milestone** Neo4j Browser shows an actual dependency graph of a real repo with visibly meaningful structure (you can point at a node and say "this file is imported by 12 others"). `POST /repos` with a JWT creates a row in Postgres. Refresh-token flow works via curl/Postman.

---

## Phase 2 — Core Engine v1 (Weeks 3–5)

### Week 3

**Person A — Deliverables**

- Extend tree-sitter parsing to **TypeScript/JS** (second language) — same output shape as Python parser (imports, functions, classes), proving the abstraction from Week 1 generalizes. Refactor parser into a `LanguageParser` interface with `PythonParser`/`TypeScriptParser` implementations.

- Graph algorithms v1 (pure Python/networkx, or Neo4j GDS — your choice, document which and why in ADR-0003): fan-in/fan-out per file, cycle detection (import cycles), top-N "hotspot" candidates by fan-in.
- Output: a `GraphMetrics` JSON report per repo (per-file fan-in/fan-out, list of detected cycles).

**Person B — Deliverables**

- `services/orchestration` skeleton (Node/TS): consumes from RabbitMQ, on message creates/updates `analysis_jobs` row.
- GitHub webhook receiver endpoint on gateway (`POST /webhooks/github`) — verifies signature, parses `push` event payload, publishes a message to RabbitMQ with `{repo_id, commit_sha}`.
- Local testing: use `ngrok` (or similar) + a real test GitHub repo to confirm a push triggers the webhook → message lands in RabbitMQ (verify via RabbitMQ management UI).
- Celery worker skeleton (Python, in `services/analysis-engine` or new `services/worker`): consumes job messages, currently just logs and updates job status to `running` then `completed` (no real work yet).

**Integration Point** This is the first **message contract** between the two halves: the JSON shape of the RabbitMQ message B's webhook publishes must be exactly what A's Celery worker (Week 4+) will consume to invoke the parser/graph code. Agree on this shape now (ADR-0004) — `{job_id, repo_id, repo_url, commit_sha, callback: "postgres"}` — even though the worker doesn't call A's code yet.

**Git Workflow**

- Branches: `feat/a-typescript-parser`, `feat/a-graph-metrics`, `feat/b-webhook-receiver`, `feat/b-orchestration-skeleton`, `feat/b-celery-worker-skeleton`.
- ~3 PRs each, <400 lines.
- Cross-review: A reviews ADR-0004 (message contract) closely — A is the eventual consumer. B reviews A's `LanguageParser` interface design for extensibility (does adding a 3rd language later look easy from the outside?).

**Cross-Train Tickets**

- A: write a small RabbitMQ consumer test script (Python `pika`) that just prints messages B's webhook publishes — confirms the contract works end-to-end from A's side, reviewed by B.
- B: run A's `GraphMetrics` CLI against a repo with a known import cycle (intentionally construct one) and confirm it's detected — reviewed by A.

**Learning Checklist**

- A: tree-sitter TS/JS grammar quirks (3 hrs); cycle detection algorithm (Tarjan's SCC or networkx `simple_cycles`) (2–3 hrs).
- B: RabbitMQ exchanges/queues/routing keys (2–3 hrs); GitHub webhook signature verification (1 hr); Celery task basics (2 hrs).

**Demo-able Milestone** A push to a test GitHub repo triggers a webhook → message visible in RabbitMQ → Celery worker picks it up and updates job status in Postgres to "completed" (even though it does no real analysis yet — the *plumbing* works). Separately, `GraphMetrics` CLI run on a TS repo and a Python repo both produce sensible fan-in/fan-out reports, and a deliberately-constructed cycle is correctly detected.

---

**Week 4**

**Person A — Deliverables**

- Git churn analysis: mine `git log --numstat` (via GitPython or subprocess) for the target repo, compute per-file: total commits touching it, lines changed, and a recency-weighted churn score (e.g., exponential decay by commit age).
- Centrality via Neo4j GDS: PageRank over the import graph (proxy for "structurally important" files). Document the GDS projection/algorithm call in ADR-0005.
- Merge churn + centrality + Week 3's fan-in/fan-out into a single per-file `RawMetrics` record, written back as Neo4j node properties.

**Person B — Deliverables**

- Wire the Celery worker to **actually invoke A's pipeline**: on job message, `git clone` the repo (shallow clone at the given commit SHA) into a scratch dir, call `analysis_engine` parse → graph → metrics functions, catch/log errors, update job status accordingly (`completed`/`failed` with error message stored).
- Frontend skeleton: `frontend/` with Vite + React + TS, routing (React Router), a basic layout (sidebar + main panel), and a "Repos" page that lists repos from `GET /repos` (Week 2 endpoint) with a "Trigger Analysis" button (calls a new gateway endpoint that publishes to RabbitMQ).
- New gateway endpoint: `POST /repos/:id/analyze` → publishes the job message (manual trigger, alternative to webhook).

**Integration Point This is the first real end-to-end wiring**: clicking "Trigger Analysis" in the frontend → gateway publishes message → orchestration creates job row → Celery worker clones repo and runs A's full Week 1–4 pipeline → writes metrics to Neo4j → job marked `completed`. The frontend doesn't display results yet, but the *pipeline* runs start-to-finish. This is a critical integration test — budget real time for it (see Risk section).

**Git Workflow**

- Branches: `feat/a-churn-analysis`, `feat/a-graph-centrality`, `feat/b-worker-pipeline-integration`, `feat/b-frontend-skeleton`, `feat/b-trigger-endpoint`.
- ~3 PRs each. `feat/b-worker-pipeline-integration` will likely be the largest/riskiest PR of the week — flag it for review early (draft PR on day 2–3, not day 7).
- Cross-review: B reviews A's `RawMetrics` schema (it's what B's worker will pass through/store). A reviews B's worker integration PR specifically for **how errors from A's code are caught and surfaced** (does a parser crash on a weird file kill the whole job, or get isolated?).

**Cross-Train Tickets**

- A: add a minimal Cypress/Playwright-free smoke check — a script that hits the frontend's "Repos" page via curl/fetch and confirms it renders the repo list (basic frontend debugging exposure) — reviewed by B.
- B: write one new churn-related Cypher query (e.g., "top 5 files by churn × fan-in" — a proto-hotspot query) — reviewed by A.

**Learning Checklist**

- A: GitPython / git log parsing (2–3 hrs); Neo4j GDS PageRank setup (2–3 hrs); exponential decay weighting design (1 hr).
- B: shallow git clone in a worker context + cleanup (1–2 hrs); React Router + Vite project structure (2–3 hrs); error propagation patterns in Celery (2 hrs).

**Demo-able Milestone** From the frontend, click "Trigger Analysis" on a registered repo → watch (via logs/Postgres) the job go `pending → running → completed`, and afterward Neo4j contains a graph with every file annotated with fan-in, fan-out, churn score, and PageRank. You can narrate: "this is our raw signal layer — next week we turn this into one score."

---

**Week 5**

**Person A — Deliverables**

- **Health Score v1**: design and implement the scoring formula combining normalized complexity (cyclomatic complexity per function, computed via tree-sitter — new sub-task), churn, coupling (instability = fan-out/(fan-in+fan-out)), and test coverage gap into a weighted per-file score (0–100), aggregated to repo-level (weighted by LOC). Weights live in a config file (not hardcoded) — sets up Week 9 tuning.
- Coverage ingestion: parse a `coverage.xml` (Cobertura/coverage.py format) for the target repo and join coverage % per file into the score.
- ADR-0006: the scoring formula, with rationale for each weight and what each component is meant to capture.

- Health scores written to Postgres (not just Neo4j) — new table `health_scores` (job_id, file_path, score, complexity, churn, coupling, coverage, computed_at).

**Person B — Deliverables**

- Frontend: first real graph visualization using cytoscape.js — render the Neo4j dependency graph (fetched via a new gateway endpoint `GET /repos/:id/graph`) with node size/color mapped to fan-in and a basic force-directed or dagre layout.
- Frontend: "Health" panel showing the repo-level health score and a sortable table of per-file scores (fetched via `GET /repos/:id/health`).
- Gateway endpoints: `GET /repos/:id/graph` (proxies a Cypher query to Neo4j), `GET /repos/:id/health` (queries Postgres `health_scores`).

**Integration Point — Phase 2 Milestone (Major) This is the first fully end-to-end product demo.** Trigger analysis from the frontend → pipeline runs (parse, graph, metrics, health score) → frontend graph view renders the actual dependency graph with visual hotspot indicators → health table shows real per-file scores. This is the moment the "system comes alive." Treat this as a real milestone: both of you should sit down together when wiring `GET /repos/:id/graph` and `GET /repos/:id/health`, since A defines the data shape and B consumes it — pair on this rather than handing off blind.

**Git Workflow**

- Branches: `feat/a-health-score-v1`, `feat/a-coverage-ingestion`, `feat/b-graph-viz`, `feat/b-health-panel`, `feat/b-graph-health-endpoints`.
- ~3 PRs each.
- Cross-review: A reviews the `GET /repos/:id/graph` endpoint's Cypher query (B wrote it, but A understands the schema). B reviews ADR-0006 (scoring formula) — push back on anything that seems arbitrary; "why is complexity weighted higher than churn?" is exactly the kind of question an interviewer will ask, so it should survive a peer challenge now.

**Cross-Train Tickets**

- A: spend 1–2 hrs in the frontend adjusting the cytoscape node coloring logic to use health score instead of fan-in (small CSS/JS-in-TS change) — reviewed by B.
- B: add cyclomatic complexity as a new field to one of A's existing test fixtures and confirm A's calculator returns the expected value — reviewed by A.

**Learning Checklist**

- A: cyclomatic complexity computation from an AST (2–3 hrs); coverage.xml format (1 hr); weighted aggregation design (1 hr).
- B: cytoscape.js basics + layouts (3–4 hrs); data-fetching patterns (React Query or SWR recommended) (1–2 hrs).

**Demo-able Milestone** A live, narratable demo: "We point Sentinel at a repo, click analyze, and within [N] seconds we get a dependency graph where node size reflects how central a file is, plus a Health Score with a breakdown — this score is something we designed ourselves, here's the formula and why." This is genuinely interview-ready as a standalone artifact even if nothing else were built.

**Depth Check (AI-shortcut risk):** A naive AI pass would produce a health score that's just `(complexity + churn) / 2` with no justification. The depth is entirely in **ADR-0006** — make sure it documents *why* each weight, what failure modes the formula has (e.g., a brand-new file with no churn history scores artificially well — is that correct?), and that this is explicitly framed as "v1, to be tuned against real data in Week 9."

---

## Phase 3 — AI Layer (Weeks 6–8)

### Week 6

#### Person A — Deliverables

- Chunking strategy: use tree-sitter boundaries (from Weeks 1/3) to split each file into function/class-level chunks with metadata (file path, name, line range, language).
- Generate embeddings for all chunks using sentence-transformers (recommend a code-aware model, e.g., a CodeBERT variant or `jina-embeddings-v2-base-code`) — benchmark embedding time on a medium repo.
- Stand up Qdrant (add to docker-compose), create a collection, batch-upsert embeddings with metadata as payload.
- ADR-0007: chunking strategy + model choice + tradeoffs (embedding cost/time vs. chunk granularity).

#### Person B — Deliverables

- WebSocket layer: add Socket.io (or native `ws`) to the gateway; Celery worker publishes job-progress events to Redis pub/sub at each pipeline stage (`parsing`, `graph_building`, `metrics`, `health_score`, `embedding`); gateway subscribes to Redis and relays to connected frontend clients via WebSocket.
- Frontend: replace polling (if any) with live job-progress UI — a progress stepper showing the current pipeline stage in real time during analysis.
- Add `analysis_jobs.current_stage` column (Postgres) updated by the worker at each stage transition.

**Integration Point** The pipeline stage names B's progress UI displays must match the actual stage names A's worker code emits — agree on a fixed enum of stage names (ADR-0008, short) now that A is adding a new "embedding" stage to the pipeline.

#### Git Workflow

- Branches: `feat/a-code-chunking`, `feat/a-embeddings-qdrant`, `feat/b-websocket-progress`, `feat/b-frontend-progress-ui`.
- ~2–3 PRs each.
- Cross-review: B reviews A's chunking metadata schema (B's frontend may eventually link findings back to specific chunks). A reviews B's Redis pub/sub → WebSocket relay for correctness under concurrent jobs (does job A's progress leak to a client watching job B?).

**Cross-Train Tickets**

- A: emit the new "embedding" stage progress event from the worker code (small addition to B's Week-6 mechanism) — reviewed by B.
- B: write a tiny script that queries Qdrant directly (`client.search(...)`) for the nearest neighbors of one arbitrary chunk, and eyeball whether the results look semantically related — reviewed by A.

**Learning Checklist**

- A: sentence-transformers usage + batching for throughput (2–3 hrs); Qdrant collection design + payload filtering (2–3 hrs).
- B: Redis pub/sub (1–2 hrs); WebSocket connection lifecycle + room/channel isolation per job (2–3 hrs).

**Demo-able Milestone** Trigger an analysis and watch a live progress stepper advance through real pipeline stages in the UI in real time (not a fake progress bar). Separately, a script demonstrates that semantically similar code chunks are nearest neighbors in Qdrant — "type hints function" and "another type hints function" cluster together even with different variable names.

---

**Week 7**

**Person A — Deliverables**

- Hybrid RAG retrieval: implement BM25 over the same chunk corpus (using `rank_bm25` or similar, in-memory or persisted), combine with Qdrant vector search via reciprocal rank fusion (RRF) to produce a single ranked chunk list for a query.
- `/ask` capability: given a natural-language question + repo_id, run hybrid retrieval, assemble top-k chunks into a prompt, call an LLM (Claude API) to generate a grounded answer with citations to `file:line`.
- ADR-0009: why hybrid (vector alone misses exact-identifier matches like function names; BM25 alone misses semantic paraphrase) — with a concrete example query that demonstrates the difference.

**Person B — Deliverables**

- New `health_score_history` table (job_id, repo_id, computed_at, repo_level_score) — populated automatically each time the pipeline completes, enabling a real trend line as more analyses run.
- Frontend: health-score trend chart (recharts) on the repo dashboard, plotting `repo_level_score` over time across multiple completed jobs.
- Frontend: scaffold the "Ask Sentinel" UI — a chat-style input box + answer display + source-citation list, wired to a new gateway endpoint `POST /repos/:id/ask` (which for now can return a stub response).
- To make the trend chart meaningful with limited real history, write a small script to re-run analysis against 3–5 historical commits of a test repo (seeding multiple `health_scores` rows with different `computed_at`/commit_sha).

**Integration Point** A finishes the real `/ask` logic (analysis-engine side) while B builds the gateway proxy + UI against a stub — by end of week, wire them together: `POST /repos/:id/ask` on the gateway calls into A's RAG service/function and returns the real grounded answer + citations to the frontend.

**Git Workflow**

- Branches: `feat/a-bm25-hybrid-retrieval`, `feat/a-ask-endpoint`, `feat/b-health-score-history`, `feat/b-trend-chart`, `feat/b-ask-ui-stub`.
- ~3 PRs each.
- Cross-review: A reviews B's historical-seeding script (does re-running analysis at old commits produce sane, comparable scores?). B reviews A's RRF combination logic and the citation format (does `file:line` map cleanly to something the frontend can link/highlight?).

**Cross-Train Tickets**

- A: wire up the frontend "Ask Sentinel" input to call the real `/ask` endpoint end-to-end (small TS/React change) — reviewed by B.
- B: write 3 test questions against a known repo and manually judge whether the hybrid retrieval result quality differs from vector-only (toggle a flag A exposes) — reviewed by A, feeds ADR-0009's example.

**Learning Checklist**

- A: reciprocal rank fusion (1–2 hrs); prompt design for grounded Q&A with citations (2–3 hrs).
- B: recharts time-series charts (2 hrs); designing a chat-style UI component (2 hrs).

**Demo-able Milestone** A live trend chart showing the health score changing across several historical commits of a real repo — "you can see this score dropped after this

commit, which is when [X] happened." Plus: type a natural-language question like "where is authentication handled in this codebase?" into "Ask Sentinel" and get a grounded answer citing specific files/lines.

---

**Week 8**

**Person A — Deliverables**

- Multi-agent review system via LangGraph, three specialist agents + synthesizer:
  - **Security agent**: runs Semgrep (with a curated ruleset) against the repo, parses findings, LLM summarizes/prioritizes by severity.
  - **Architecture-smell agent**: queries graph metrics from Weeks 3–5 (high fan-in/fan-out outliers, cycles, "god classes" by LOC+method count) and has the LLM explain *why* each is a smell in plain language.
  - **Test-gap agent**: cross-references coverage data + centrality — flags high-centrality, low-coverage files as risk, LLM suggests what kinds of tests would help.
  - **Synthesizer**: LangGraph node taking all findings as shared state, deduplicating overlapping findings, and computing a **confidence score** per finding (function of agent agreement + underlying metric confidence — document the formula).
- ADR-0010: agent architecture diagram (LangGraph graph structure) + confidence-weighting formula.

**Person B — Deliverables**

- `agent_findings` table (Postgres): job_id, agent_type, file_path, severity, message, confidence, created_at.
- Worker integration: after the main pipeline completes, trigger the agent graph (A's code) as a subsequent stage (`agent_review`), store results in `agent_findings`.
- Frontend "Findings" tab: list findings grouped by file or by agent, with severity badges (color-coded) and confidence scores displayed, sortable/filterable.

**Integration Point** This is the second major integration point: the `agent_findings` schema (B) must accommodate whatever shape A's synthesizer outputs (one finding = one agent + file + message + severity + confidence — confirm this is sufficient before B builds the table, ideally before mid-week).

**Git Workflow**

- Branches: `feat/a-langgraph-agents`, `feat/a-synthesizer`, `feat/b-agent-findings-schema`, `feat/b-agent-pipeline-stage`, `feat/b-findings-ui`.
- This is likely the **largest PR set of the project** — A should split into at least 4 PRs (one per agent + synthesizer) rather than one giant PR, to keep reviews tractable.

- Cross-review: B reviews each agent PR primarily for **output shape consistency** (does every agent emit the same finding structure, regardless of internal logic?) — this is reviewable without deep LangGraph knowledge. A reviews B's `agent_review` worker stage for failure isolation (if Semgrep isn't installed/crashes, do the other two agents still run?).

**Cross-Train Tickets**

- A: add a `severity` filter dropdown to B's Findings UI (small React change, using the enum A defined) — reviewed by B.
- B: read through the synthesizer's confidence-weighting code and write 2–3 example findings by hand to sanity-check the formula produces sensible scores (e.g., does an issue flagged by 2 agents score higher than one flagged by 1?) — reviewed by A, becomes part of ADR-0010.

**Learning Checklist**

- A: LangGraph state graphs, conditional edges, shared state design (4–6 hrs — this is the steepest curve of the project); Semgrep ruleset basics (2 hrs); prompt design for multiple specialist roles (2–3 hrs).
- B: designing a schema flexible enough for heterogeneous agent outputs without becoming a generic blob (1–2 hrs); severity-based UI patterns (1–2 hrs).

**Demo-able Milestone** Run analysis on a real repo and show the Findings tab populated with concrete, specific findings — e.g., "Security agent flagged a hardcoded secret in `config.py` (high severity, confidence 0.9)" and "Architecture-smell agent flagged `utils.py` as a god module with fan-in of 23 and 1,400 LOC (medium severity, confidence 0.7)" — with the synthesizer's reasoning visible.

**Depth Check (AI-shortcut risk):** This is the week most tempting to let an LLM "wing it" — three agents that just call an LLM with slightly different prompts and call it a day is shallow. The genuine engineering is in: (1) the LangGraph **state/control flow** (how findings accumulate, how agents can run in parallel where independent), (2) the **confidence formula** being a real function of structured data (agent agreement, metric magnitude) rather than the LLM just inventing a number, and (3) **failure isolation** (one agent's LLM call failing shouldn't break the others). Make sure ADR-0010 demonstrates all three.

---

## Phase 4 — Integration & Hardening (Weeks 9–10)

**Week 9**

**Person A — Deliverables**

- Run the full pipeline end-to-end against 2–3 real OSS repos of increasing size (small ~50 files, medium ~300–500 files, and — if time allows — large ~1,500+ files). Log and fix crashes/edge cases encountered (binary files, generated code, unusual encodings, deeply nested directories).
- Tune Health Score weights (ADR-0006 config) based on whether scores "make sense" against these real repos — does a file you both independently judge as messy actually score poorly? Document tuning decisions and remaining known limitations honestly.
- Tune agent confidence weighting based on observed false-positive patterns (e.g., if the architecture-smell agent flags every test file as a god module, adjust).

**Person B — Deliverables**

- Add Prometheus client instrumentation to gateway, orchestration, and worker: request counts/latencies, queue depth, job duration histograms, job success/failure counters.
- Stand up Prometheus + Grafana (docker-compose), build 2–3 dashboards: pipeline throughput, job latency breakdown by stage, error rates.
- Begin CI/CD: GitHub Actions workflow that builds Docker images for all services on merge to `main` and pushes to a registry (GitHub Container Registry is simplest).

**Integration Point** A's larger-repo test run is also B's first real load test of the queue/worker system under non-trivial duration — run it together and watch the new Grafana dashboards live. If the large-repo run reveals the pipeline takes too long or fails, that's this week's most important finding — see Risk section.

**Git Workflow**

- Branches: `fix/a-large-repo-edge-cases` (expect several small fix PRs, <150 lines each — this is bugfix week, not feature week), `tune/a-health-score-weights`, `feat/b-prometheus-instrumentation`, `feat/b-grafana-dashboards`, `feat/b-ci-image-builds`.
- Cross-review: both of you should review each other's bugfix PRs lightly but quickly — this week prioritizes velocity over deep review, since fixes tend to be small and isolated.

**Cross-Train Tickets**

- A: add one Prometheus metric of your own choosing to the analysis-engine worker (e.g., "embeddings generated per second") and confirm it appears in Grafana — reviewed by B.
- B: read through ADR-0006's updated weights and the tuning notes; write up, in your own words, what you'd want to test next if you had more real repos — reviewed by A, becomes part of the "future work" section of the final README.

**Learning Checklist**

- A: handling encoding/binary-file edge cases robustly (1–2 hrs); systematic tuning methodology (documenting before/after) (1–2 hrs).
- B: Prometheus metric types (counter/histogram/gauge) (2–3 hrs); Grafana dashboard JSON/PromQL basics (2–3 hrs); GHCR image publishing from Actions (1–2 hrs).

**Demo-able Milestone** A Grafana dashboard showing real pipeline metrics from a run against a 500+ file open-source repo — "here's our p95 job latency broken down by stage, here's queue depth over time." Plus: a documented before/after comparison of Health Score weights showing a deliberate tuning decision with reasoning.

---

**Week 10**

**Person A — Deliverables**

- Performance pass on the analysis-engine for larger repos: batch Neo4j writes more aggressively if needed, consider streaming/incremental parsing if the large-repo run from Week 9 was too slow, add a configurable file-size/LOC cap with graceful skipping (and a "skipped files" report) for pathological files.
- Embedding generation performance: batch size tuning, consider caching embeddings for unchanged files between runs (using file hash) — a real optimization with a clear "before/after" story.
- ADR-0011: performance findings — what was the bottleneck, what was done about it, what's left as known limitation.

**Person B — Deliverables**

- Full cloud deployment: k3s cluster (or AWS ECS — pick based on what's more learnable for you; k3s is cheaper/more portable for a portfolio demo) running all services + Postgres + Neo4j + Redis + RabbitMQ + Qdrant.
- Secrets management (k8s Secrets or ECS equivalent) — no credentials in images/configs.
- Grafana accessible externally (or via port-forward for demo purposes); confirm dashboards work against the deployed environment.
- Basic load test (k6 or Locust) against the gateway/webhook endpoint — document results.

**Integration Point** Deploy and run a full end-to-end analysis against the **deployed** environment, not just locally. This surfaces integration issues that only appear under real network boundaries (service discovery, env var config, timeouts) — budget significant time for this, it rarely goes smoothly first try.

**Git Workflow**

- Branches: `perf/a-incremental-embeddings`, `perf/a-large-file-handling`, `feat/b-k3s-deployment`, `feat/b-secrets-management`, `feat/b-load-testing`.
- Deployment config PRs (`infra/` changes) should be reviewed by **both** of you regardless of who owns what — infra config affects everyone's ability to develop/demo.

**Cross-Train Tickets**

- A: SSH/kubectl into the deployed cluster and manually trigger a job via the deployed gateway's API (no UI) — confirms A understands the deployment topology — reviewed by B.
- B: review A's "skipped files" report format and add a small UI element (Findings tab or job summary) surfacing it — reviewed by A.

**Learning Checklist**

- A: file-hash-based caching strategy (1–2 hrs); batching/streaming tradeoffs for large data (2 hrs).
- B: k3s basics (pods, services, deployments, secrets) (4–6 hrs — significant if new); k6/Locust basics (2 hrs).

**Demo-able Milestone** A live demo against a deployed (not localhost) environment: trigger analysis from the deployed frontend, watch it complete, view results — with a Grafana dashboard open showing the deployed system's metrics in real time. This is the "it's actually a deployed system" milestone that matters a lot in interviews.

**Depth Check (AI-shortcut risk):** AI-generated k8s manifests are extremely common and easy to spot as copy-paste ("why does this Deployment have 3 replicas for a stateful Neo4j pod?"). The depth here is in **deliberate resource sizing decisions** and **documenting why** certain services are/aren't horizontally scaled (e.g., "the worker is scaled to 2 replicas because jobs are CPU-bound during embedding generation; Neo4j and Postgres are single-replica because we're not running this as a multi-tenant production system"). Write this reasoning into ADR-0011 or a new ADR-0012.

---

## Phase 5 — Polish, Docs & Interview Prep (Weeks 11–12)

**Week 11**

**Person A — Deliverables**

- Architecture diagrams: data-flow diagram (Mermaid) showing the full pipeline from webhook → parse → graph → metrics → embeddings → agents → frontend; a separate diagram for the LangGraph agent structure.

- Write/finalize `services/analysis-engine/README.md` covering setup, design decisions (linking to relevant ADRs), and known limitations.
- **Final Swap (capstone cross-training):** pick one meaningful improvement to B's frontend or gateway code (e.g., improve error states in the Findings UI, add a loading skeleton, fix a UX rough edge you noticed during demos) and implement it yourself — reviewed by B.

**Person B — Deliverables**

- Top-level `README.md`: project pitch, architecture diagram (using A's Mermaid diagrams), quickstart (`docker-compose up`), tech stack table, links to ADRs, demo video/gif placeholder.
- Record the demo video (5–7 min walkthrough script — write the script first, then record): trigger analysis → graph view → health score + trend → Ask Sentinel → findings.
- **Final Swap (capstone cross-training):** pick one meaningful improvement to A's analysis-engine code (e.g., a missing edge-case test, a refactor for clarity in a confusing module, an additional Cypher query for a useful metric) and implement it yourself — reviewed by A.

**Integration Point** Both Final Swap PRs should target real, useful improvements (not token changes) — review each other's swap PR as if it were a normal contribution, including pushing back if something's off. This is deliberately the most "interchangeable" week of the project.

**Git Workflow**

- Branches: `docs/architecture-diagrams`, `docs/readme-main`, `swap/a-frontend-improvement`, `swap/b-analysis-engine-improvement`.
- Fewer, more carefully-written PRs this week — docs PRs in particular benefit from both of you reading closely.

**Cross-Train Tickets**

- (Subsumed by the Final Swap deliverables above — no separate tickets this week.)

**Learning Checklist**

- A: Mermaid diagram syntax (1 hr); technical writing for a general audience (ongoing).
- B: screen-recording + basic video editing for the demo (2–3 hrs); technical writing.

**Demo-able Milestone** A polished top-level README that someone unfamiliar with the project could read in 5 minutes and understand what Sentinel does, how it's built, and why key decisions were made — plus a recorded demo video.

---

**Person A — Deliverables**

- Final bug-fixing pass based on demo-video rehearsal (run the demo script live 2–3 times, fix anything that breaks or looks rough).
- Resume bullets (3–4), each tied to a specific technical decision with a measurable or demonstrable claim (e.g., "Designed a hybrid retrieval pipeline combining BM25 and vector search via reciprocal rank fusion, improving retrieval relevance for exact-identifier queries").
- Mock interview prep: 10–15 questions you should be able to answer in depth about your half (e.g., "Walk me through what happens if the Semgrep process crashes mid-run" / "Why PageRank over betweenness centrality for hotspot detection?").

**Person B — Deliverables**

- Same final bug-fixing pass, from the platform/infra side (deployment stability, websocket reconnection edge cases, etc.).
- Resume bullets (3–4) tied to specific decisions (e.g., "Built an event-driven analysis pipeline using RabbitMQ/Celery with real-time job-progress relay via Redis pub/sub and WebSockets").
- Mock interview prep: 10–15 questions about your half.

**Both — Shared Deliverables**

- 5–10 **cross-cutting** mock interview questions both of you should be able to answer (since the interview narrative is "we built this together and understand the whole system") — e.g., "How does a finding's confidence score get computed, end to end?" or "If you had another month, what would you build next and why?"
- Tag a `v1.0` release.
- Final deployment health check.

**Integration Point** A final full end-to-end run, ideally against a repo neither of you has tested against before, as a genuine "does this hold up" check.

**Git Workflow**

- Branches: `fix/final-polish-*` (small, targeted fixes), `docs/resume-and-interview-prep` (this can be a private doc in `docs/`, not necessarily in the public README).
- Light review load — this week is about stabilization, not new features.

**Learning Checklist**

- Both: practicing explaining the *other* person's architectural decisions out loud (1–2 hrs each, using the ADRs and shared cross-cutting questions as a script).

---

## Highest-Risk Weeks & Contingency Plans

### 1. Week 4 — Worker/Pipeline Integration + Graph Algorithms + Churn

**Why it's risky:** This is the first time A's analysis code, B's worker/queue plumbing, and a real git clone all have to work together — plus A is simultaneously learning Neo4j GDS and churn mining. Integration bugs here tend to be subtle (path resolution, working directory issues, dependency conflicts between the worker's environment and analysis-engine's).

**Contingency:**

- If GDS-based centrality (PageRank) is the blocker, **drop it for now** and ship with just fan-in/fan-out (already built in Week 3) — PageRank becomes a Week 9 stretch addition once the pipeline is stable.
- If churn analysis with recency-weighting is taking too long, ship **v0 churn = raw commit count per file** (a 30-minute implementation) and explicitly note in ADR-0006 that recency-weighting is a planned refinement.
- If the worker integration itself is the blocker, **pair on it** (Swap-Sprint-style, even though not officially scheduled) rather than letting it block both of you separately — this is the one integration point where a few hours of pairing saves days of async back-and-forth.

### 2. Week 8 — Multi-Agent System

**Why it's risky:** Highest learning curve (LangGraph), most open-ended ("when is an agent good enough?"), and the schema/contract with B isn't fully knowable until A has built at least one agent end-to-end.

**Contingency:**

- **De-scope to two agents**: keep the **security agent** (most concrete — Semgrep output is structured, low ambiguity) and **architecture-smell agent** (leverages graph work already done, high "wow factor" for a demo). Cut or defer the **test-gap agent** — its insight (high-centrality + low-coverage = risk) can actually be expressed as a derived metric on the existing Health Score / Findings UI without a dedicated LLM agent, framed honestly as "v2: a dedicated test-gap agent with its own reasoning."
- **Simplify the synthesizer**: if an LLM-based reconciliation step isn't converging, replace it with a **rule-based confidence formula** (e.g., `confidence = base_confidence_by_agent × agreement_bonus_if_multiple_agents_flag_same_file`)

— still a "real algorithm we designed," just not LLM-based. Document this as a deliberate choice in ADR-0010, not a failure.

- If even two agents is too much by mid-week, ship the **security agent alone** first as a complete vertical slice (LangGraph node → findings table → UI), then add the second agent in whatever time remains — a working 1-agent system beats a broken 3-agent system for a demo.

### 3. Week 10 — Cloud Deployment

**Why it's risky:** Deployment is notorious for "works on my machine" failures, and k3s/ECS both have real learning curves; this week also depends on Week 9's performance work being done, which can slip.

**Contingency:**

- If full k3s/ECS deployment is too much, **fall back to a single-VM Docker Compose deployment** (e.g., a $5–10/month VPS running the same `docker-compose.yml` you've used locally all along) — this is still "deployed and demo-able," just not "orchestrated." Frame this honestly: "we deployed via Compose for the demo environment and have a documented path to k3s (here's the manifest skeleton) as the next step" — interviewers respect an honest "here's what we'd do for real production scale" answer.
- If the large-repo performance work from Week 9 isn't done, **demo against the medium-sized repo only** for Week 10's milestone, and explicitly carry large-repo performance into Week 11/12 polish (it's lower-stakes there since it's not blocking other work).
- Decouple Prometheus/Grafana from the deployment if needed — a locally-running Grafana pointed at a deployed system's `/metrics` endpoints (via port-forward or public endpoint) is an acceptable fallback to "fully deployed monitoring stack."

---

## Repo Structure & README Strategy

**Monorepo** (recommended): for a 2-person team with this many interdependent services, a monorepo gives you a single CI pipeline, single source of truth for ADRs/docs, and a much better first impression for someone browsing the repo (one `git clone`, one `docker-compose up`, everything's there).

```
sentinel/
├── README.md                        # top-level pitch, architecture,
quickstart
├── docs/
│   ├── architecture/                # Mermaid diagrams (system, data flow,
agent graph)
│   ├── adr/                          # 0001-repo-analysis-job-contract.md,
0002-..., etc.
│   └── demo/                          # demo script, screenshots, video link
├── services/
│   ├── analysis-engine/             # Person A — Python: parsing, graphs,
scoring
│   ├── agents/                      # Person A — LangGraph multi-agent
system (or subpackage of above)
│   ├── gateway/                     # Person B — Fastify API gateway +
webhooks + WebSocket
│   ├── auth/                        # Person B — auth service
│   └── orchestration/               # Person B — job orchestration + Celery
worker glue
├── frontend/                         # Person B — React/TS dashboard
├── infra/
│   ├── docker-compose.yml
│   ├── k3s/  (or ecs/)              # deployment manifests
│   ├── prometheus/
│   └── grafana/
├── .github/workflows/
└── scripts/                          # dev setup, seed data, local test
repos
```

**ADR template** (keep these short — half a page each):

```markdown
# ADR-000X: <Title>
**Status:** Proposed | Accepted | Superseded by ADR-00YY
**Context:** What problem are we solving?
**Decision:** What did we choose?
**Alternatives considered:** What else, and why not?
**Consequences:** What does this make easier/harder later?
```

**README strategy:**

- Top-level README: pitch (2–3 sentences on the real-world problem), architecture diagram, "Quick Start" (`docker-compose up`, then what to do), tech stack table, links to each service README and to `docs/adr/`, demo video embed.

- Each service README: setup/dev instructions, a short "Design Notes" section linking to relevant ADRs, and a brief "Owned primarily by [Person], reviewed by [Person]" line — honest about ownership while signaling the collaboration model.
- A short "How We Worked" section in the top-level README (a paragraph) describing the cross-review + cross-training process — this is a differentiator interviewers notice, since most portfolio projects from pairs/groups don't actually show evidence of real collaboration.

---

### Weekly Check-In Format (For This Chat)

Paste this at the start of each week's check-in; I'll use it to assess drift and propose re-plans if needed.

```markdown
## Week N Check-In

**Planned vs. Done**
- A: [item] — ✅ done / ⚠️ partial / ❌ not started — [1-line note if not ✅
- A: ...
- B: [item] — ✅ / ⚠️ / ❌ — [note]
- B: ...

**Integration Point Status**
- Did this week's integration point actually connect end-to-end? Yes / Partia
- [1-2 lines on what happened]

**Cross-Train Ticket**
- A's ticket: done? what was learned/surprising?
- B's ticket: done? what was learned/surprising?

**Blockers / Surprises**
- [anything unexpected — new dependency, harder-than-expected concept, scope

**Carry-Over**
- [items moving to next week]

**Re-plan Needed?**
- Yes/No — if yes, what's the proposed adjustment? (I'll help work through de
```

This format is intentionally light — the goal is a 5-minute weekly sync, not a status report. If a week goes sideways, flag it early (mid-week is fine) rather than waiting for the check-in — the contingency plans above assume early detection.